

Plan

PowerPulse — Build Plan

Written this morning as “Umuriro”; the name was settled at 14:20. Kept as the plan of record, not rewritten after the fact — where the build diverged from it, the build is what shipped.

Team: Token Titans (5 people) **Event:** Frontiers GenAI Hackathon, ALX Kigali, 28 July 2026 **Track:** 04 — Energy & Climate Infrastructure (also fully satisfies Track 03 — see `CONTEXT.md` §3 for why we file under 04) **Hard deadline:** 4:30pm submission window. Code freeze 3:00pm.

1. What we are building

A Kinyarwanda-first web app (mobile viewport) that answers one question for a person whose power just went out:

“Is this planned, how long is it likely to last, and what should I do about it right now?”

The third part is the product. Anyone can look up a schedule. The value is situation-specific advice that changes based on **how long the outage will last** and **who the user is**.

Why generative AI is load-bearing

Rehearse this answer — a judge will ask.

1. **Messy, code-switched Kinyarwanda input** matched against free-text place names (“nta muriro mu Kimironko”, “power out Gisozi”, “hano mu Masaka nta amashanyarazi”).
2. **Unstructured source text** — REG’s `Sector / Areas` and `Reason` are free text with inconsistent formatting, multiple districts per row, feeder names embedded in prose.
3. **Generated advice** conditioned on duration + user profile. This is not a lookup; the same location at the same moment produces materially different guidance for different users.

What we are NOT building

- No crowdsourced outage reporting. No user-submitted data of any kind.
- No dashboard for REG or any planning authority.
- No household device inventory / energy-usage survey.
- No native mobile app.

These were all considered and cut deliberately. They have cold-start problems, deliver value to someone who is not in the room, and do not demo. **If someone on the team proposes adding one of these mid-day, the answer is no.**

2. Data source (verified, real, public)

URL: <https://www.reg.rw/customer-service/power-outages/>

An HTML table with these columns:

Date | Time | District Affected | Sector / Areas | Reason | Status |

- Status is one of Planned, Current, Past.
- **126 pages of pagination** — years of historical records.
- Pagination via query param: ?
`tx_poweroutage_poweroutagelist%5B%40widget_0%5D%5BcurrentPage%5D=N`
- Filter routes also exist: `/listing/{month}/` and `/listing/{statusId}/` where status 1=Current, 2=Planned, 3=Past.

Sample rows actually observed:

```
05 Aug 2026 | 12:00 PM - 02:00 PM | Gasabo | Gisozi, Kinyinya & Kacyiru | Maintenance and extension works on "UTEXRWA" feeder | Planned
04 Aug 2026 | 12:00 PM - 02:00 PM | Karongi, Rutsiro & Nyarugenge | Rubengera, Gitesi, Bwishyura, Rwankuba, Mutuntu in Karongi; All sectors in Rutsiro; Gitega, Rwezamenyo, Nyakabanda, Kimisagara, Nyarugenge in Nyarugenge | Maintenance and extension works on "Kibuye" & "Nyamirambo" feeders | Planned
03 Aug 2026 | 12:00 PM - 02:00 PM | Bugesera, Kicukiro, Gasabo & Rwamagana | Mwogo, Juru, Nyamata in Bugesera; Masaka in Kicukiro; Rusororo in Gasabo; Nyakariro in Rwamagana | Maintenance and extension on works "Mwogo" feeder & "Masaka" feeder "Masaka" T-off | Planned
```

Note the format variance: multiple districts per row, X, Y in District; Z in OtherDistrict, All sectors in Rutsiro, feeder names in quotes inside Reason.

Secondary source (optional, only if time permits): REG posts unplanned outage notices on X/Twitter (@reg_rwanda) in Kinyarwanda and English, usually with a cause and no ETA. Do not build a Twitter integration. If you want unplanned examples for the demo, hand-copy 5–10 into a JSON file and label them clearly as such.

3. TASK ZERO — scrape before anything else

This runs in the first 30 minutes. Nothing else starts until the CSV is on disk.

If reg.rw rate-limits, goes down, or blocks us at 2pm, the whole project dies unless we already have the data locally. Get it now.

```
scripts/scrape_reg.py
```

Requirements: - Iterate pages 1..126 via the `currentPage` param. - Parse the outage table with BeautifulSoup (`lxml` parser). - Polite delay (~1s) between requests. - Write raw rows to `data/outages_raw.json` with fields exactly as scraped, plus `page_number` and `source_url`, under a wrapper carrying `source_url / scraped_at / record_count`. - Commit the JSON to the repo. Yes, really — this is a hackathon and reproducibility matters more than repo hygiene. Judges opening the repo should see the data. - On any HTTP error, retry twice then log and continue. Do not let one bad page kill the run.

Pagination gotcha (verified, not theoretical): the `currentPage` param requires a matching `cHash` . Without it the site returns **HTTP 200 serving page 1** — no error. Iterating 1..126 naively yields 126 copies of page 1 and looks like it worked. The scraper therefore follows the “Next” link chain and fingerprints each page to prove it is new. Also: `/listing/{N}/` is a **month** filter, not pagination.

Acceptance: `data/outages_raw.json` exists, a random sample of 5 records visually matches the website, and `page_count` is 126. Note the table serves **4 rows per page**, so the realistic total is ~500 records, not >1000.

4. Data model and parsing

```
scripts/parse_outages.py → data/outages.json
```

Target schema, one row per (outage × affected sector):

FIELD	TYPE	NOTES
<code>outage_id</code>	str	hash of raw row
<code>date</code>	ISO date	parse “05th August 2026” → 2026-08-05
<code>start_time</code>	HH:MM	from “12:00 PM - 02:00 PM”
<code>end_time</code>	HH:MM	may be null for unplanned
<code>duration_minutes</code>	int	null if end unknown
<code>district</code>	str	one per row — explode multi-district rows
<code>sector</code>	str	one per row — explode the sector list
<code>reason_raw</code>	str	raw text
<code>feeder</code>	str	extract quoted names from reason, e.g. UTEXRWA, Masaka
<code>reason_category</code>	enum	maintenance / extension / fault / vandalism / other
<code>status</code>	enum	planned / current / past

Parsing rules: - `Sector / Areas` splits on `;` for district groups, then `,` and `&` within a group. The trailing in `<District>` assigns the district. - All sectors in `X` → emit a single row with `sector = "*"` meaning district-wide. Handle the wildcard in matching. - Normalize sector names: lowercase, strip accents/whitespace, for the matching index only. Keep the original for display. - Non-parseable rows go to `data/parse_failures.json` . Do not silently drop them; the count is worth mentioning in the README.

Acceptance: `data/outages.json` exists, exploded records > raw records, `parse_failures.json` is under ~5% of input.

5. Historical statistics

```
src/stats.py
```

This is our differentiator. Nobody else will have real historical grounding.

Functions, all computed from `outages.json` :

- `outages_for_sector(sector, district=None)` → all matching records
- `outage_count(sector, window_days=365)` → int
- `mean_duration(sector)` → minutes, and the sample size it's based on
- `typical_hours(sector)` → most common start-hour bucket
- `feeder_for_sector(sector)` → most frequently associated feeder name
- `historical_estimate(sector)` → returns `{estimate_minutes, n, confidence}` where confidence is low if $n < 5$

Hard rule: every stat returns its sample size `n`. The LLM must be told `n` and must not present a low-`n` estimate as a firm prediction. Small samples are the single most likely source of a misleading output in this project.

6. Location matching

```
src/locate.py
```

Input: free-text Kinyarwanda or English user message. Output: best-matching `(district, sector)` plus a confidence.

Approach — do the cheap thing first: 1. Normalize the input. 2. Direct substring match against the sector/district index built from `outages.json`. 3. Fuzzy match (`token_set_ratio`) with a threshold around 80. 4. Only if 1–3 fail, ask the LLM to extract a place name from the message, then re-run 1–3 on the extracted name.

If we cannot match with confidence, ask the user which sector they mean — offer the 3 closest candidates. Do not guess. A wrong location produces a confidently wrong answer, which is the worst failure mode we have.

7. User profiles

Three, fixed. Not “etc.” — three.

PROFILE	WHAT IT CHANGES
household	food safety, lighting, phone charging, water pump if applicable
shop_owner	cold stock decisions, generator economics, whether to move inventory
remote_worker	work continuity, battery budget, whether relocating is worth it

Captured at first use with three taps (or a spoken sentence if voice lands). Store in `localStorage`. No accounts, no backend user table.

Optional profile detail, only if free: for `household` and `shop_owner`, an optional short list of cold items. Voice or text. Feeds the food module (§9).

8. Answer generation

```
src/answer.py
```

Pipeline:

```
user message
→ locate (§6)
→ query current/planned outages for that sector
→ historical_estimate (§5)
→ build context object
→ LLM call
→ structured answer
```

Context object passed to the model:

```
{
  "sector": "Gisozi",
  "district": "Gasabo",
  "now": "2026-07-28T14:12:00+02:00",
  "matched_outage": {
    "status": "planned",
    "start": "12:00",
    "end": "14:00",
    "reason": "Maintenance and extension works on \"UTEXRWA\" feeder",
    "minutes_remaining": -12
  },
  "history": {
    "count_365d": 7,
    "mean_duration_minutes": 121,
    "n": 7,
    "confidence": "medium",
    "typical_hour": "12:00"
  },
  "profile": "shop_owner",
  "cold_items": ["inyama", "amata", "ice cream"]
}
```

If `matched_outage` is null → this is the **unplanned / no-ETA path**, which is the more interesting demo case.

System prompt constraints (non-negotiable)

Write these into the prompt explicitly, and keep them in the repo as `prompts/system.txt` so judges can read them:

1. **Answer in Kinyarwanda by default.** Mirror the user's language if they wrote in English.
2. **Never state a restoration time that REG has not published.** If there is no published end time, say clearly that REG has not given one, then give the historical average *labelled as historical*, with its sample size.
3. **Never invent facts about the world.** No named cafés, shops, or neighbourhoods that “have power.” No claims about the cause beyond what `reason_raw` says.
4. **Advice must reference the actual duration.** If duration is unknown, the advice must be framed around uncertainty. No duration → no confident advice.
5. **Bias conservative on food, stock, and money.** When a call is marginal, recommend the safe option (discard, don't risk the stock). State that we bias toward waste over illness.
6. **If sample size is below 5, say the estimate is weak.** Do not launder a statistic into a promise.
7. **If the location cannot be matched, ask — do not guess.**

Output shape

Ask for JSON, parse it, render it. Fields:

```
{
  "status_line": "...",
  "duration_line": "...",
  "advice": ["...", "..."],
  "confidence_note": "...",
  "source": "REG, planned outage, 05 Aug 2026"
}
```

The `source` line renders under every answer in the UI. Visible grounding is worth more in a 2-minute video than any amount of visual polish.

9. Food safety module (build only after §1–§8 work end to end)

Thresholds live in `data/food_safety.json`, **not** in the model's head. The LLM explains and personalizes; it does not calculate.

Baseline (conservative, unopened): - Refrigerator holds safe temperature ~4 hours. - Full freezer ~48 hours; half-full ~24 hours. - Every door opening reduces this — tell the user to keep it shut. - Above threshold → discard. No “probably fine.”

Note the important framing point: **planned outages here are consistently 2 hours, which is below the spoilage threshold.** So the honest answer for most planned outages is “*your food is fine, don't open the fridge.*” The food module earns its place in the **unplanned, long-duration** case. Demo it there.

10. Evaluation set (do not skip — this is our score)

```
eval/questions.json
eval/run_eval.py → eval/results.md
```

30 questions in Kinyarwanda: - 25 answerable from REG data (mix of sectors, mix of planned/past, some with messy or code-switched phrasing) - 5 deliberately out of scope (locations not in the data, or questions we cannot answer — “when will electricity get cheaper?”)

Three scored metrics:

METRIC	WHAT IT MEASURES
Location match accuracy	correct (district, sector) extracted — target ≥ 24/25
Correct refusal rate	out-of-scope questions where the system says it doesn't know — target 5/5
Duration-grounded advice	on 5 selected questions, did the advice actually use the correct duration — target 5/5

Output a markdown table. **These numbers go in the video, the README, and the 1-pager.** Most submissions will claim their tool works; ours will show it.

11. Web app

- Mobile viewport, **390px**, styled as an SMS/chat thread. Message bubbles, one input, one send button.
- **Plain HTML + JS is fine and probably faster than React.** Do not spend time on a framework.
- Source line rendered under every answer.
- Profile selector: three buttons on first load, stored in `localStorage`.
- **Deploy an empty “hello” page to Vercel in the first 15 minutes** with the repo connected. Getting deployment failures out of the way early is worth more than it sounds.

Say this explicitly in the video and 1-pager: “*The production channel is SMS/USSD. This web interface is the demo surface.*” That converts our only apparent weakness — a web app for a low-connectivity user — into evidence we understood the constraint.

Optional, only if one person gets a message round-tripping in 45 minutes: Africa’s Talking Rwanda sandbox for real SMS. **If it isn’t working in 45 minutes, kill it.** It is a classic hackathon time sink and is a nice bullet, not a nice demo.

12. Priority stack — ship in this order, cut from the bottom

1. Location match + is there an outage, planned or not, with the published window ← **must ship**
2. Historical pattern for that area, with sample size
3. Honest no-ETA behavior + duration-conditioned advice by profile ← **this is the differentiator**
4. Food safety module
5. Voice input (Kinyarwanda)
6. SMS integration

If item 3 is not working by 2:00pm, cut 4, 5 and 6 entirely.

13. Schedule

TIME	WHAT
now → +30min	scraper runs, <code>outages_raw.csv</code> on disk. Vercel hello-world deployed.
+30min → 12:30	parsing + stats in parallel with pipeline/prompt work. Target: one hardcoded question → one correct grounded answer.
12:30	first end-to-end run, however ugly. Artifacts owner records a rough backup video now.
12:30 → 2:00	Kinyarwanda output quality, no-ETA path, profile-conditioned advice, history in answers.
2:00 → 3:00	eval set built, UI cleaned enough to film.
3:00	HARD CODE FREEZE.
3:00 → 4:15	real video, README, 1-pager.
4:15	submit. Not 4:29.

14. Team split (5)

ROLE	OWNS
Data	scraper, parser, <code>outages.csv</code> , stats functions
Pipeline	LLM integration, context building, prompt engineering, JSON parsing
Kinyarwanda + Eval	output quality in Kinyarwanda, builds and scores the 30-question set
Interface	the 390px web app — the thing that appears in the video
Artifacts	video, README, 1-pager, pitch. Not on the critical path for code. Joins eval after lunch.

The Artifacts owner is assigned **at the start of the day**, not at 3pm. Their job at 1pm is to notice that nothing is recordable yet — which is exactly when there's still time to fix it.

15. Submission artifacts

2-minute video

TIME	CONTENT
0:00–0:15	The problem, concrete and local. Power's out; nobody knows if it's planned or how long.
0:15–0:20	One sentence on what we built.
0:20–1:20	The working thing. Real question in Kinyarwanda, real answer, source line visible.
1:20–1:40	The money shot: same location, same moment, two profiles → two different answers. Proves it isn't a lookup table.
1:40–1:50	Data provenance (REG, N records) + eval numbers on screen as text.
1:50–2:00	Honest-uncertainty case: no ETA published, system says so instead of inventing one. Close.

No team intro. No logo animation. No slide preamble. Product visible by 0:20. Record voiceover separately from the screen capture — room noise kills more hackathon videos than bad content. Burn in subtitles; judges may watch muted.

GitHub repo

Judged by the README, not the code. Assume nobody runs it.

README order: one-line description → GIF/screenshot of it working → problem → architecture diagram → **data source with the reg.rw link** → eval results table → ethics section → setup steps.

Include `prompts/system.txt` in the repo. Spread commits through the day rather than one 4pm dump.

1-pager

A scanning document, not an essay. Bold headers, short blocks, numbers. A judge should read it in 40 seconds and know whether to advance us.

Sections: Problem · What it does · Why generative AI is load-bearing · Data source · Eval results · Ethics choices · What's next.

16. Ethics bonus mapping (state this explicitly in the 1-pager)

The sheet awards extra credit for a human approval step, evaluation metrics, and personalization with constraints. All three are structural here, not bolted on:

- **Human approval step** — the system asks the user to confirm the location when match confidence is low, rather than guessing. Advice is framed as guidance with sources shown, never as an instruction to follow blindly.
- **Evaluation metrics** — \$10. Three scored metrics including a refusal rate.

- **Personalization with constraints** — §7 and §8. Advice varies by profile *and* by duration, under explicit constraints: no invented restoration times, no invented facts about the world, conservative on food and money, weak estimates labelled as weak.

Best line for the judges: “We built it to say ‘I don’t know.’” Most teams would hallucinate an ETA to make the demo smooth. Show a real no-ETA case on purpose.

17. Known pitfalls

- **The “why genAI?” question.** Have the §1 answer ready in one breath.
 - **Small-sample statistics.** A sector with 2 historical outages will produce a confident-sounding average. Guard it with `n` and confidence labels.
 - **Sector name collisions.** Some sector names repeat across districts (e.g. Nyarugenge is both a district and a sector). Always carry the district.
 - **Food safety over-confidence.** Real illness risk. Conservative defaults, thresholds in data not in the prompt.
 - **Scope creep back toward the dashboard.** See §1. The answer is no.
 - **Leaving artifacts to the end.** The 3:00 freeze is the whole plan. Defend it.
-

Appendix — model selection (verified 2026-07-28)

Don’t trust the model list from `ListModels` — if you copy a tutorial written for `gemini-2.5-flash` or `gemini-2.0-flash` it will fail with a confusing 404 rather than an auth error. Use the 3.x models. **gemini-3.6-flash is the sensible default: fast, cheap, 1M context.**

Verified working on our key: `gemini-3.6-flash`, `gemini-3.5-flash`, `gemini-3.5-flash-lite`, `gemini-3.1-pro-preview`, `gemini-3.1-flash-lite`, `gemini-flash-latest`, `gemini-pro-latest`, `gemini-embedding-2` (3072-dim).

Listed but dead (404 on call): `gemini-2.5-flash`, `gemini-2.5-pro`, `gemini-2.0-flash`, `gemini-3-pro-preview`.

Key is in `.env` as `GOOGLE_GENERATIVE_AI_API_KEY`.